

運用設計

目次

1 進捗の確認と操作: WBS サイト	4
2 レンダリング規則(重要)	4
3 承認ゲートの運用	4
4 ISSUE 運用	4
5 ⑤ fail の裁定と復帰	5
6 CLI ドライバ(pipeline.py)	6
7 実行手順(ランブック)	6
8 オプション一覧	8
9 設計上の性質	8
10 レビューアーへの配布(単体実行 EXE)	12
11 PDF 出力(種別ごとの合本)	12

人(オペレータ)の仕事は4つに限定される — トリガを引く・質問に答える・承認する・裁定する。コードや文書を直接書く必要は基本的にない(書いてよい場所は後述)。この4つはチャットからでもブラウザからでも行える。

運転モードは5経路。規模だけでなく「どこまで自動で進むか」が違う。

表1

モード	規模の目安	進む範囲	起動	レビューの単位
対話	1件ずつ	1関数 × 1工程	<code>/legacy-N-xxx F-xxxx</code>	1件ずつチャットで
会話バッチ	～数十件	①をN件	「仕様書をまとめて10件進めて」	一斉レビュー表でまとめて
ブラウザ単発	1件ずつ	1関数 × 1工程	仕様書ページのボタン	その場のウィジェットで
ブラウザ連続実行	数十～数百件	①～⑤を工程横断で自動	バッチ実行状況ページ	同じ画面の「人待ち」から
無人バッチ (CLI)	数百～2000件	①のみ・全件	<code>pipeline.py spec</code> (夜間実行)	翌朝、一斉レビュー表で

- どのモードでも承認が人であることは変わらない。粒度と自動化範囲の違いだけ
- ブラウザ連続実行と CLI 無人バッチは同じ実行枠を共有するので、二重に走らせても片方が待たされるだけで壊れない
- CLI の `pipeline.py` が①専用なのに対し、ブラウザ連続実行は②以降も進む。「①を全件先に流したい」なら CLI、「関数単位で完成まで持っていきたい」なら連続実行

導入手順

対象プロジェクト側のセットアップ(初回のみ):

1. skill の配置

```
cp -r skills/legacy-reverse <project>/.claude/skills/  
cp -r skills/legacy-reverse/skills/* <project>/.claude/skills/
```

(2 行目は skill 発見のため。フェーズ skill を .claude/skills/ 直下にも置く)

2. **hook の登録(必須)** — `legacy-reverse/hooks/settings-example.json` を `<project>/.claude/settings.json` にマージ。2 本入っている: `guard_tests.py`(`PreToolUse`。④⑤中の `tests/` 編集を拒否)と `guard_json.py`(`PostToolUse`。壊れた JSON をエージェントに差し戻す)。運用中のプロジェクトに後から入れる場合は `PostToolUse` ブロックの追記だけでよい
3. **MCP サーバの登録(推奨)** — `<project>/mcp.json` に `legacy-reverse-mcp` を追加 (MCP サーバ仕様)。未登録でも動作は同じ
4. **ツール類** — Quarto(未導入なら `quarto-typst-pdf skill` の `qtpdf.py install` でポータブル導入・管理者権限不要)、Chromium 系ブラウザ(PDF の Mermaid 焼き込み用。HTML だけなら不要)、`pip install pytest sphinx sphinx-rtd-theme`(⑦では追加で `radon ruff bandit pip-audit`)
5. **開始** — `/legacy-reverse` でセットアップ状態を確認 → 新規なら `/legacy-0-analyze`

1 進捗の確認と操作: WBS サイト

進捗の確認だけでなく、実行・承認・裁定もこのサイトで完結する (各フェーズの最後に自動更新される)。

```
python <LR>/scripts/serve_site.py --root .
```

- 127.0.0.1 のみに bind(仕様書は社内資料。LAN 公開は `--host 0.0.0.0` を明示したときだけ)。LAN 公開しても書き込み系の操作はローカルからのみ受け付ける
- ポートはプロジェクト名から決まる固定値(8100-8899)。URL をブックマークできる。複数プロジェクト同時でも衝突しない(埋まっていれば空きへずらす)
- キャッシュ無効で配信。再レンダリング後はブラウザ更新だけで最新
- `--render` で配信前に作り直し、`--watch` で docs/ 変更を検知して自動再レンダリング。ただしバッチ実行状況ページは Quarto を通さないで `--watch` に関係なくリアルタイム
- ナビバー: WBS / ドメイン知識 / 規約 / ⑥完了検証 / ⑦分析 / 新コード詳細(API) / バッチ状況。一斉レビュー表は navbar になく、WBS の「要対応」からリンクする

画面ごとの構成・ボタンの出現条件・HTTP API の詳細は 画面設計。

2 レンダリング規則(重要)

- 各フェーズの最後に必ず `ledger wbs → render_site.py` を実行する契約 (人がブラウザで途中経過を常に確認できる状態を保つ)
- `quarto render docs` を直接叩かない。Quarto は Mermaid を .qmd でしか描けないため、`render_site.py` が影コピー(docs/_sitework/)を作ってから render する
- 成果物(.md)の図は GitHub 流の ```mermaid.` ```mermaid` と書くと サイト全体の render が落ちる
- Sphinx(④ API)は必ず `render_site.py` の後(render_site が _site を作り直すため)。MCP の `render_site` ツールはこの正順を内包している

3 承認ゲートの運用

対象: ①の reviewed 化、②の approved 化、⑤トリアージの (b)(c)、ループ上限後の再開。承認の経路は 2 系統あるが、記録先も検証も同じ。

表 3-1

経路	流れ
チャット	skill が承認用サマリ(変更点・Confidence 内訳・未回答質問)を提示 → 人が OK / 修正指示 / 保留 → skill がフロントマターを更新
ブラウザ	仕様書ページの承認ウィジェット、または一斉レビュー表の行内ボタン → サーバが機械レビューを再実行してからフロントマターを更新 → WBS・一斉レビュー表・サイトを再生成

- 勝手に approved にしない。承認待ちで turn を終えるのは正しい動作
- どちらの経路でも記録先は同じフロントマター(reviewed-by / reviewed-date、②は approved-*)
- 答えられない質問は「保留」でよい。ISSUE として WBS 最上部に出続ける
- draft は書き直し自由。reviewed の書き直しは②の再承認が必要になる旨を先に確認される
- ブラウザからの「修正依頼」は docs/review-feedback.md に pending として溜まり、次に AI が①②を実行したときに自動で拾われて applied になる

4 ISSUE 運用

- 採番は全体通し(ledger next-issue)。形式は「仮説+Yes/No で答えられる問い」を強制(白紙の質問は来ない。人の判断コストを下げる)
- kind: spec-gap(④が詰まった) / domain / legacy-bug / triage(⑤ループ上限) / other

- 回答は2系統どちらでも同等: チャットで回答 or ISSUE ファイルの「回答(人が記入)」欄に直接記入
- 回答が付いたら answered → 成果物へ反映して applied。ドメイン知識は domain-knowledge.md に転記され、同じ質問は二度と来ない
- 全フェーズ skill は起動時に「回答済み ISSUE・規約変更」をスキャンしてから本処理に入る (ファイルに直接書いた内容は次のフェーズ起動時に自動で拾われる)

5 ⑤ fail の裁定と復帰

表 5-1

分類	意味	人の操作
(a) 実装バグ	実装が①を満たしていない	何もしない(AI が④修正 → ⑤再実行を自走)
(b) テストコードバグ	③が②とズレている	ISSUE の証拠を見て承認 → ③再生成
(c) 仕様が誤り	②①自体が間違い	ISSUE で裁定 → ①改訂を指示(伝搬は自動検知)

(a) のループは 3 回で自動停止し、triage ISSUE 起票+WBS 🚫。復帰手順は 2 通り。

チャット経路(3 ステップ):

1. ISSUE を読んで (a)/(b)/(c) を裁定し回答(チャットまたは回答欄)
2. AI が反映(applied)したら unblock(「F-xxxx をアンブロックして」でよい)
3. /legacy-5-test F-xxxx を再トリガ(attempt は 1 からやり直し)

ブラウザ経路(1 操作 + 1 クリック): 裁定待ちの関数の仕様書ページ、またはバッチ実行状況ページの「人待ち」タブに ISSUE の質問がそのまま出る。回答を書いて送ると「ISSUE への記入(answered)→ unblock → サイト更新」まで完了し、リロード後に出る「⑤を実行する」ボタンで再テストできる。

無人バッチ実行

全件実行(数百～2000 件)はエージェントの会話内ループでは行わない(コンテキスト上限・コンパクション劣化)。無人ドライバを使う。入口は2つある。

表 5-2

入口	カバー範囲	件数指定	コスト上限	向く場面
CLI <code>pipeline.py spec</code>	①のみ	<code>--max-funcs</code>	<code>--budget-usd</code>	夜間に①を全件流す
ブラウザ「連続実行」	①～⑤	プルダウン(全部 / 1 / 10 / 50 / 100)	UIからは指定しない	承認しながら関数を完成まで進める

どちらも実行の実体(`run_one`)と実行枠ロックを共有するので、同時には走らない。ブラウザ側でコスト上限を外したのは、件数指定で足りる上に、コストは画面の指標カードで常時見えているため(画面設計)。

6 CLI ドライバ(`pipeline.py`)

```
python <LR>/scripts/pipeline.py spec --root . # 全件 draft まで無人実行
python <LR>/scripts/pipeline.py spec --root . --max-funcs 200 # 今日は 200 件だけ
python <LR>/scripts/pipeline.py spec --root . --budget-usd 20 # コスト上限
python <LR>/scripts/pipeline.py spec --root . --dry-run # 対象の確認のみ
```

7 実行手順(ランブック)

エージェント(headless Claude)を機械的に回すための通し手順。

1. 前提を確認する

- `claude --version` が通ること(PATH に無ければ `--claude-cmd <パス>` で明示)
- 対象プロジェクトに skill 一式が配置済みであること(導入手順 1～2)
- **headless は許可プロンプトに答えられない。**①が使うツール(Read/Write/Edit と スクリプト実行)を `.claude/settings.json` の `permissions.allow` で許可しておくか、信頼できる閉じた環境に限り `--skip-permissions` (= `--dangerously-skip-permissions`)を明示する

2. 対象と実行内容を確認する(`dry-run`)

```
python <LR>/scripts/pipeline.py spec --root . --dry-run
```

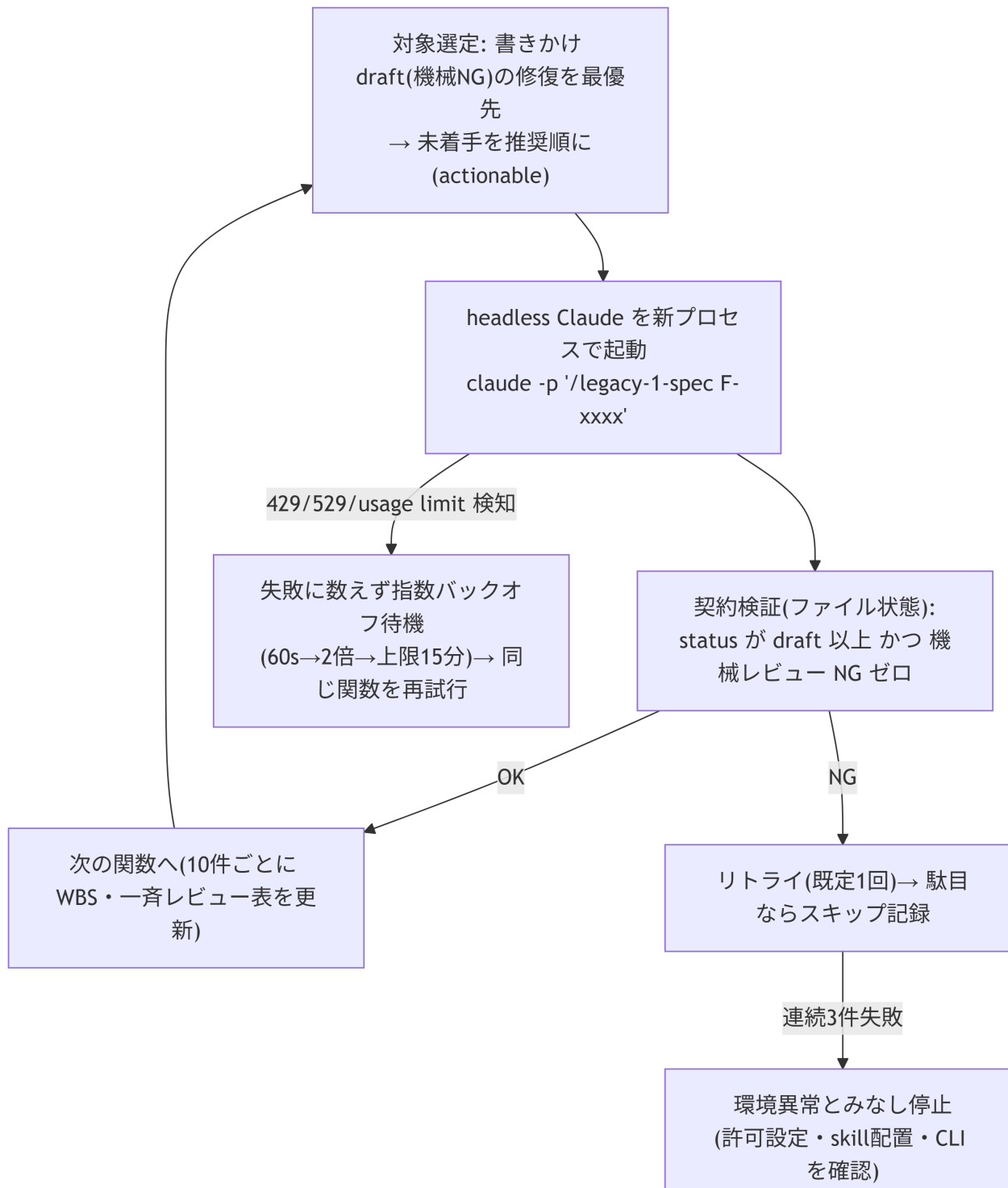
対象件数と、1件ごとに起動される `claude` コマンドがそのまま表示される(既定: `claude -p "/legacy-1-spec F-xxxx" --output-format json --max-turns 50`)

3. 起動する

```
python <LR>/scripts/pipeline.py spec --root . [--max-funcs N] [--budget-usd X] [--model M]
```

4. ドライバが1関数ごとに行うこと(人の介在なし):

図 7-1



5. 監視する(任意。放置してよい)

- 端末出力: [n] F-xxxx draft 化 OK(累計 \$X.XX) / 検証 NG・待機の理由
- .legacy-reverse/pipeline-log.jsonl: 1 行 1 関数(結果・attempt・所要秒・コスト)
- ブラウザ: serve_site.py --watch を立てておけば WBS と一斉レビュー表が チャンク ごとに自動更新される

6. 止まる条件(いずれも安全停止。再開は同じコマンド)

- --max-funcs / --budget-usd の上限到達
- 連続失敗が --max-consecutive-fail(既定 3)に到達 = 環境異常の疑い
- レート待機の累計が --rate-wait-total(既定 6 時間)を超過

7. 中断・再開 — Ctrl-C・電源断はどこでも安全。同じコマンドで続きから再開 (処理済み draft は再選定されず、書きかけ draft は機械 NG として先に修復される)
8. 翌朝の人の作業 — docs/spec-review.md(一斉レビュー表)を見て 「全部 OK / F-xxxx は修正: ~」を返す → reviewed 化。失敗として スキップされた関数はログを確認して個別に `/legacy-1-spec F-xxxx` に対応

8 オプション一覧

表 8-1

オプション	既定	意味
<code>--max-funcs N</code>	0(無制限)	このセッションで処理する上限件数
<code>--budget-usd X</code>	0(無制限)	累計コスト上限。超えたら停止
<code>--chunk N</code>	10	WBS・一斉レビュー表を更新する間隔
<code>--retries N</code>	1	契約検証 NG 時のリトライ回数
<code>--max-consecutive-fail N</code>	3	連続失敗でドライバを停止する閾値
<code>--max-turns N</code>	50	1 関数あたりのエージェントターン上限
<code>--timeout N</code>	1800	1 関数あたりの秒数上限(ハング対策。超過は通常リトライ経路)
<code>--backoff-base / --backoff-max</code>	60 / 900	レートリミット検知時の待機秒(初期値→2 倍ずつ→上限)
<code>--rate-wait-total N</code>	21600	レート待機の累計上限秒。超えたら安全停止
<code>--pause N</code>	0	関数間の予防的待機秒(レートリミットに当たりやすい環境用)
<code>--model M</code>	—	claude に渡すモデル指定
<code>--skip-permissions</code>	off	<code>--dangerously-skip-permissions</code> を渡す(信頼できる環境のみ)
<code>--claude-cmd / --claude-args</code>	PATH 検索 / —	claude 実行ファイルの明示 / 追加引数
<code>--prompt-template</code>	<code>/legacy-1-spec {fid}</code>	1 関数あたりのプロンプト(<code>{fid}</code> が置換される)
<code>--dry-run</code>	off	実行せず対象とコマンドを表示
<code>--no-render</code>	off	終了時の HTML 再生成を省略

9 設計上の性質

- 1 関数 = 1 つの新しい headless Claude プロセス(`claude -p "/legacy-1-spec F-xxxx"`)。コンテキストが積み上がり、1 関数あたりのトークンは常に一定 — トークン上限に依存しない
- 完了は LLM の申告でなく **ファイル状態で契約検証**(status: draft + 機械レビュー NG ゼロ)。NG はリトライ → スキップ記録、連続失敗で安全停止
- **タイムアウト・レートリミット耐性**: 1 関数ごとに秒数上限で打ち切り(ハング対策)。レートリミット / 利用枠上限は失敗に数えず指数バックオフで待機 → 同じ関数から自動再開 (利用枠の時間リセットを人手なしで跨げる。待機累計の上限超過で安全停止)
- チャンクごとに WBS・一斉レビュー表を自動更新。人は溜まった **draft** を**随時レビュー**してよい (全件完了を待つ必要はない)
- 中断(Ctrl-C・電源断)はどこでも安全。同じコマンドで**続きから再開** (処理済み draft は二重に書き直さない — 再開時は `next_actions(skip_draft=True)` 相当の選定)
- 実行ログ: `.legacy-reverse/pipeline-log.jsonl`(関数別の結果・コスト・所要)
- 前提: 対象プロジェクトに skill 配置済み、headless 用に必要ツールを `.claude/settings.json` で allow(または `--skip-permissions` を明示)

典型的な運用: 夜間に回して、翌朝一斉レビュー表(spec-review.md)を見る。

中断・再開

進捗の正はすべてファイル側(functions.json / ledger.json / フロントマター)にあり、会話コンテキストには無い。どのタイミングで中断しても、再開手順は常に同じ3コマンド:

```
ledger status --summary          # 全体状況(2000 関数でも数行)
ledger next --all --limit 20     # 着手可能な関数と次フェーズ
python <LR>/scripts/review_checks.py all --root .  # 成果物の健全性
```

チャットなら /legacy-reverse と打つだけ(上記を実行して次の一手を提案してくる)。

- ・やり直し・再列挙は禁止。全関数を列挙した長大な出力をコンテキストに読み込まない
- ・①の再実行も抽出器がマージ動作なので安全
- ・「どこまで進んだか分からない」ときは WBS を開くのが最短(🚫🔴🔴🔴 が「要対応」として最上部)

大規模運用(200 関数超)

- WBS のトップは自動でダッシュボードに切り替わる(進捗サマリ・要対応・人への質問・次の一手・レガシーファイル別進捗)。全関数明細は docs/wbs/ のファイル別サブページへ分割
- 2000 関数でも「いま何が問題で、次に何をやるか」はトップ 1 画面で分かる
- 状況把握は `progress_summary / next_actions`(MCP)または `summary / next`(CLI)で。全関数表を読む運用はしない

10 レビューアへの配布(単体実行 EXE)

サーバ用意や社内公開なしで、サイト同梱の実行ファイルを渡し各自のローカルホストで開く。

```
pip install pyinstaller # 初回のみ
python <LR>/scripts/build_viewer.py --root . # → dist/<プロジェクト>-wbs.exe
```

- 受け手はダブルクリックだけ。Python も Quarto も不要(サイト込み 10MB 程度)
- 外部フォント参照はビルド時に除去 — 閉域ネットワークでも開け、外部通信も飛ばない
- 中身はビルド時点のスナップショット。進捗が動いたら作り直して渡し直す(--no-render で再レンダリング省略可)
- クロスコンパイル不可: Windows 用 .exe は Windows 上でビルド
- SmartScreen 等に止められる環境は --onedir(フォルダ配布)、サイト別配布なら --no-embed

11 PDF 出力(種別ごとの合本)

```
python <LR>/scripts/pdf_book.py specs --root . --output pdf/関数仕様書.pdf --title 関数仕様書
python <LR>/scripts/pdf_book.py test-specs --root . --output pdf/テスト仕様書.pdf --title テスト仕様書
python <LR>/scripts/pdf_book.py test-results --root . --output pdf/テスト結果報告書.pdf --title テスト結果報告書
```

- quarto-typst-pdf skill に委譲(Quarto 1.10 系 book+typst の既知不具合 2 件を前処理とパッチで回避)
- PDF では相対リンク・アンカーは平文化される(リンクは HTML 版に残る)
- 生成後は qtpdf.py check <pdf> で豆腐・はみ出しを機械チェック
- Mermaid の PDF 化には Chromium 系ブラウザが要る(qtpdf.py doctor で確認)

手編集の可否

表 11-1

ファイル	手編集	説明
docs/domain-knowledge.md ・ docs/conventions.md	○	人が著者。業務知識・規約はここへ
ISSUE の「回答(人が記入)」欄	○	じっくり書きたい裁定はファイルで
docs/specs/(仕様書)	△	編集可。ハッシュ連鎖が下流を stale にして再確認が走る(設計どおり)
docs/review-feedback.md	△	ブラウザが追記し skill が applied 化する。人が直接足してもよいが書式を守る
WBS(index.qmd) ・ test-results/ ・ completion-check.md ・ spec-review.md ・ quant.md ・ perf.md	✕	自動生成。次の再生成で消える
data/functions.json	△	正データ。編集するなら JSON を壊さない こと(壊れると全スクリプトが停止する。hook が AI の編集は検査する)
data/ledger.json	✕	スクリプト専用
.legacy-reverse/ 配下すべて	✕	実行時データ。pipeline-status.json は表示専用、batch-priority.json は画面が書く

conventions.md を途中で変更した場合、skill が変更を検知して「どの関数の成果物と不整合になり得るか」を洗い出して人に報告する契約。

表 11-2

症状	意味	対処
WBS に  ISSUE-xxx	④⑤ループ上限、裁定待ち	ISSUE に回答 → unblock → ⑤再トリガ
WBS に stale△	①改訂で②が古い	/legacy-2-testspec で再生成 → 再承認
WBS に △改変	freeze 後にテストが変わった	意図した変更なら③で再freeze。心当たりがなければ調査
tests/ 編集が拒否された	hook が正常動作	テスト側の疑義は ISSUE 経由(正規経路)
再レンダリング失敗	配信サーバが _site をロック	serve_site.py を使う(cwd を動かさないでロックしない)
図がソースのまま表示	quarto render を直叩きした	render_site.py で出し直す
①②の機械レビュー NG 報告	ハルシネーション・省略を検知(正常動作)	AI が直して再提出してくる。NG 付き成果物を承認しない
⑥で完全性突合 NG	抽出器の 2 系統カウント不一致(変則ソース)	extract-report.json の該当ファイルを AI に調査させる。判断がつかなければ ISSUE
バッチが「データ異常のため停止」	正データ(functions.json 等)が JSON として壊れた	停止理由に行・列が出る。git diff で直前の変更を確認して修復、または git checkout -- で戻す。hook 登録済みならこの状態になる前に AI へ差し戻される
連続実行が「連続 3 件失敗」で停止	環境異常の疑いで安全停止(1 件ずつの失敗とは別扱い)	結果パネルの NG 行から応答ログを読む。API キー・レート・claude の解決を確認して再開始
「別の実行が進行中」と断られる	実行枠ロックを双方向で取り合う設計(正常動作)	完了を待つか停止する。異常終了の残骸なら次の実行時に自動解除される
画面がずっと「実行中」のまま	実行プロセスが異常終了した残骸	PID の死活で自動的に「停止(異常終了)」へ読み替えて表示される。そのまま再実行してよい
配布 EXE で承認ボタンが効かない	EXE はスナップショットなので書き込み系を 403 で拒否する(仕様)	生成元のプロジェクトを serve_site.py で開いて操作する
サイト内検索に新しいページが出ない	差分レンダは検索索引を更新しない	節目で render_site.py --root . --full を 1 回実行する
配布 EXE が古い	ビルド時点のスナップショット	build_viewer.py で作り直す
PDF だけ図が出ない	Mermaid 画像化に Chromium が要る	qtpdf.py doctor で確認。HTML は影響なし

より詳しい一覧は MANUAL.md のトラブルシューティング章を参照。